

LogisticRegression

```
class sklearn.linear_model.LogisticRegression(penalty='deprecated',  
*, C=1.0, l1_ratio=0.0, dual=False, tol=0.0001, fit_intercept=True,  
intercept_scaling=1, class_weight=None, random_state=None,  
solver='lbfgs', max_iter=100, verbose=0, warm_start=False,  
n_jobs=None)
```

[\[source\]](#)

Logistic Regression (aka logit, MaxEnt) classifier.

This class implements regularized logistic regression using a set of available solvers. **Note that regularization is applied by default.** It can handle both dense and sparse input **X**. Use C-ordered arrays or CSR matrices containing 64-bit floats for optimal performance; any other input format will be converted (and copied).

The solvers 'lbfgs', 'newton-cg', 'newton-cholesky' and 'sag' support only L2 regularization with primal formulation, or no regularization. The 'liblinear' solver supports both L1 and L2 regularization (but not both, i.e. elastic-net), with a dual formulation only for the L2 penalty. The Elastic-Net (combination of L1 and L2) regularization is only supported by the 'saga' solver.

For [multiclass](#) problems (whenever `n_classes >= 3`), all solvers except 'liblinear' optimize the (penalized) multinomial loss. 'liblinear' only handles binary classification but can be extended to handle multiclass by using [OneVsRestClassifier](#).

Read more in the [User Guide](#).

Parameters:

penalty : {'l1', 'l2', 'elasticnet', None}, default='l2'

Specify the norm of the penalty:

- **None** : no penalty is added;
- **'l2'** : add an L2 penalty term and it is the default choice;
- **'l1'** : add an L1 penalty term;
- **'elasticnet'** : both L1 and L2 penalty terms are added.

Warning

Some penalties may not work with some solvers. See the parameter **solver** below, to know the compatibility between the penalty and solver.

! **Added in version 0.19:** l1 penalty with SAGA solver (allowing 'multinomial' + L1)

! **Deprecated since version 1.8:** `penalty` was deprecated in version 1.8 and will be removed in 1.10. Use `l1_ratio` and `C` instead. `l1_ratio=0` for `penalty='l2'`, `l1_ratio=1` for `penalty='l1'`, `l1_ratio` set to any float between 0 and 1 for `penalty='elasticnet'`, and `C=np.inf` for `penalty=None`.

C : float, default=1.0

Inverse of regularization strength; must be a positive float. Like in support vector machines, smaller values specify stronger regularization. `C=np.inf` results in unpenalized logistic regression. For a visual example on the effect of tuning the `C` parameter with an L1 penalty, see: [Regularization path of L1- Logistic Regression](#).

l1_ratio : float, default=0.0

The Elastic-Net mixing parameter, with $0 \leq \text{l1_ratio} \leq 1$. Setting `l1_ratio=1` gives a pure L1-penalty, setting `l1_ratio=0` a pure L2-penalty. Any value between 0 and 1 gives an Elastic-Net penalty of the form $\text{l1_ratio} * \text{L1} + (1 - \text{l1_ratio}) * \text{L2}$.

Warning

Certain values of `l1_ratio`, i.e. some penalties, may not work with some solvers. See the parameter `solver` below, to know the compatibility between the penalty and solver.

! **Changed in version 1.8:** Default value changed from None to 0.0.

! **Deprecated since version 1.8:** `None` is deprecated and will be removed in version 1.10. Always use `l1_ratio` to specify the penalty type.

dual : bool, default=False

Dual (constrained) or primal (regularized, see also [this equation](#)) formulation. Dual formulation is only implemented for l2 penalty with liblinear solver. Prefer `dual=False` when $n_{\text{samples}} > n_{\text{features}}$.

tol : float, default=1e-4

Tolerance for stopping criteria.

fit_intercept : bool, default=True

Specifies if a constant (a.k.a. bias or intercept) should be added to the decision function.

intercept_scaling : *float, default=1*

Useful only when the solver `liblinear` is used and `self.fit_intercept` is set to `True`. In this case, `x` becomes `[x, self.intercept_scaling]`, i.e. a “synthetic” feature with constant value equal to `intercept_scaling` is appended to the instance vector. The intercept becomes `intercept_scaling * synthetic_feature_weight`.

Note

The synthetic feature weight is subject to L1 or L2 regularization as all other features. To lessen the effect of regularization on synthetic feature weight (and therefore on the intercept) `intercept_scaling` has to be increased.

class_weight : *dict or ‘balanced’, default=None*

Weights associated with classes in the form `{class_label: weight}`. If not given, all classes are supposed to have weight one.

The “balanced” mode uses the values of `y` to automatically adjust weights inversely proportional to class frequencies in the input data as `n_samples / (n_classes * np.bincount(y))`.

Note that these weights will be multiplied with `sample_weight` (passed through the fit method) if `sample_weight` is specified.

Added in version 0.17: `class_weight='balanced'`

random_state : *int, RandomState instance, default=None*

Used when `solver == 'sag', 'saga'` or `'liblinear'` to shuffle the data. See [Glossary](#) for details.

solver : *{‘lbfgs’, ‘liblinear’, ‘newton-cg’, ‘newton-cholesky’, ‘sag’, ‘saga’}, default=‘lbfgs’*

Algorithm to use in the optimization problem. Default is ‘lbfgs’. To choose a solver, you might want to consider the following aspects:

- ‘lbfgs’ is a good default solver because it works reasonably well for a wide class of problems.
- For [multiclass](#) problems (`n_classes >= 3`), all solvers except ‘liblinear’ minimize the full multinomial loss, ‘liblinear’ will raise an error.

- 'newton-cholesky' is a good choice for $n_samples \gg n_features * n_classes$, especially with one-hot encoded categorical features with rare categories. Be aware that the memory usage of this solver has a quadratic dependency on $n_features * n_classes$ because it explicitly computes the full Hessian matrix.
- For small datasets, 'liblinear' is a good choice, whereas 'sag' and 'saga' are faster for large ones;
- 'liblinear' can only handle binary classification by default. To apply a one-versus-rest scheme for the multiclass setting one can wrap it with the [OneVsRestClassifier](#).

Warning

The choice of the algorithm depends on the penalty chosen ($l1_ratio=0$ for L2-penalty, $l1_ratio=1$ for L1-penalty and $0 < l1_ratio < 1$ for Elastic-Net) and on (multinomial) multiclass support:

solver	l1_ratio	multinomial multiclass
'lbfgs'	$l1_ratio=0$	yes
'liblinear'	$l1_ratio=1$ or $l1_ratio=0$	no
'newton-cg'	$l1_ratio=0$	yes
'newton-cholesky'	$l1_ratio=0$	yes
'sag'	$l1_ratio=0$	yes
'saga'	$0 \leq l1_ratio \leq 1$	yes

Note

'sag' and 'saga' fast convergence is only guaranteed on features with approximately the same scale. You can preprocess the data with a scaler from [sklearn.preprocessing](#).

➡ See also

Refer to the [User Guide](#) for more information regarding [LogisticRegression](#) and more specifically the [Table](#) summarizing solver/penalty supports.

! **Added in version 0.17:** Stochastic Average Gradient (SAG) descent solver. Multinomial support in version 0.18.

! **Added in version 0.19:** SAGA solver.

! **Changed in version 0.22:** The default solver changed from 'liblinear' to 'lbfgs' in 0.22.

! **Added in version 1.2:** newton-cholesky solver. Multinomial support in version 1.6.

max_iter : *int, default=100*

Maximum number of iterations taken for the solvers to converge.

verbose : *int, default=0*

For the liblinear and lbfgs solvers set verbose to any positive number for verbosity.

warm_start : *bool, default=False*

When set to True, reuse the solution of the previous call to fit as initialization, otherwise, just erase the previous solution. Useless for liblinear solver. See [the Glossary](#).

! **Added in version 0.17:** warm_start to support lbfgs, newton-cg, sag, saga solvers.

n_jobs : *int, default=None*

Does not have any effect.

! **Deprecated since version 1.8:** n_jobs is deprecated in version 1.8 and will be removed in 1.10.

Attributes:

classes_ : *ndarray of shape (n_classes,)*

A list of class labels known to the classifier.

coef_ : *ndarray or CSR matrix of shape (1, n_features) or (n_classes, n_features)*

Coefficients of the features in the decision function.

`coef_` is of shape (1, n_features) when the given problem is binary.

By default, it will be created as a dense array, but can be turned to sparse (CSR format) through [sparsify](#) (which can be beneficial under L1 regularization when many coefficients are zero), and back to dense through [densify](#).

intercept_ : *ndarray of shape (1,) or (n_classes,)*

Intercept (a.k.a. bias) added to the decision function.

If `fit_intercept` is set to False, the intercept is set to zero. `intercept_` is of shape (1,) when the given problem is binary.

n_features_in_ : *int*

Number of features seen during [fit](#).

! Added in version 0.24.

feature_names_in_ : *ndarray of shape (n_features_in_,)*

Names of features seen during [fit](#). Defined only when `X` has feature names that are all strings.

! Added in version 1.0.

n_iter_ : *ndarray of shape (1,)*

Actual number of iterations for all classes.

! Changed in version 0.20: In SciPy <= 1.0.0 the number of lbfgs iterations may exceed `max_iter`. `n_iter_` will now report at most `max_iter`.

➡ See also

[SGDClassifier](#)

Incrementally trained logistic regression (when given the parameter `loss="log_loss"`).

[LogisticRegressionCV](#)

Logistic regression with built-in cross validation.

Notes

The underlying C implementation uses a random number generator to select features when fitting the model. It is thus not uncommon, to have slightly different results for the same input data. If that happens, try with a smaller tol parameter.

Predict output may not match that of standalone liblinear in certain cases. See [differences from liblinear](#) in the narrative documentation.

References

L-BFGS-B – Software for Large-scale Bound-constrained Optimization

Ciyou Zhu, Richard Byrd, Jorge Nocedal and Jose Luis Morales.

<http://users.iems.northwestern.edu/~nocedal/lbfgsb.html>

LIBLINEAR – A Library for Large Linear Classification

<https://www.csie.ntu.edu.tw/~cjlin/liblinear/>

SAG – Mark Schmidt, Nicolas Le Roux, and Francis Bach

Minimizing Finite Sums with the Stochastic Average Gradient <https://hal.inria.fr/hal-00860051/document>

SAGA – Defazio, A., Bach F. & Lacoste-Julien S. (2014).

[“SAGA: A Fast Incremental Gradient Method With Support for Non-Strongly Convex Composite Objectives”](#)

Hsiang-Fu Yu, Fang-Lan Huang, Chih-Jen Lin (2011). Dual coordinate descent

methods for logistic regression and maximum entropy models. Machine Learning 85(1-2):41-75. https://www.csie.ntu.edu.tw/~cjlin/papers/maxent_dual.pdf

Examples

```
>>> from sklearn.datasets import load_iris
>>> from sklearn.linear_model import LogisticRegression
>>> X, y = load_iris(return_X_y=True)
>>> clf = LogisticRegression(random_state=0).fit(X, y)
>>> clf.predict(X[:2, :])
array([0, 0])
>>> clf.predict_proba(X[:2, :])
array([[9.82e-01, 1.82e-02, 1.44e-08],
       [9.72e-01, 2.82e-02, 3.02e-08]])
>>> clf.score(X, y)
0.97
```

For a comparison of the LogisticRegression with other classifiers see: [Plot classification probability](#).

decision_function(*X*)

[\[source\]](#)

Predict confidence scores for samples.

The confidence score for a sample is proportional to the signed distance of that sample to the hyperplane.

Parameters:

***X* : {array-like, sparse matrix} of shape (n_samples, n_features)**

The data matrix for which we want to get the confidence scores.

Returns:

scores : ndarray of shape (n_samples,) or (n_samples, n_classes)

Confidence scores per (n_samples, n_classes) combination. In the binary case, confidence score for `self.classes_[1]` where >0 means this class would be predicted.

densify()

[\[source\]](#)

Convert coefficient matrix to dense array format.

Converts the `coef_` member (back) to a numpy.ndarray. This is the default format of `coef_` and is required for fitting, so calling this method is only required on models that have previously been sparsified; otherwise, it is a no-op.

Returns:

self

Fitted estimator.

fit(*X*, *y*, *sample_weight*=None)

[\[source\]](#)

Fit the model according to the given training data.

Parameters:

***X* : {array-like, sparse matrix} of shape (n_samples, n_features)**

Training vector, where `n_samples` is the number of samples and `n_features` is the number of features.

***y* : array-like of shape (n_samples,)**

Target vector relative to *X*.

sample_weight : array-like of shape (n_samples,) default=None

Array of weights that are assigned to individual samples. If not provided, then each sample is given unit weight.

! **Added in version 0.17:** *sample_weight* support to LogisticRegression.

Returns:

self

Fitted estimator.

Notes

The SAGA solver supports both float64 and float32 bit arrays.

get_metadata_routing()

[\[source\]](#)

Get metadata routing of this object.

Please check [User Guide](#) on how the routing mechanism works.

Returns:

routing : *MetadataRequest*

A [MetadataRequest](#) encapsulating routing information.

get_params(deep=True)

[\[source\]](#)

Get parameters for this estimator.

Parameters:

deep : *bool, default=True*

If True, will return the parameters for this estimator and contained subobjects that are estimators.

Returns:

params : *dict*

Parameter names mapped to their values.

predict(X)

[\[source\]](#)

Predict class labels for samples in X.

Parameters:

X : *{array-like, sparse matrix} of shape (n_samples, n_features)*

The data matrix for which we want to get the predictions.

Returns:

y_pred : *ndarray of shape (n_samples,)*

Vector containing the class labels for each sample.

predict_log_proba(X)

[\[source\]](#)

Predict logarithm of probability estimates.

The returned estimates for all classes are ordered by the label of classes.

Parameters:

X : *array-like of shape (n_samples, n_features)*

Vector to be scored, where *n_samples* is the number of samples and *n_features* is the number of features.

Returns:

T : *array-like of shape (n_samples, n_classes)*

Returns the log-probability of the sample for each class in the model, where classes are ordered as they are in *self.classes_*.

predict_proba(X)

[\[source\]](#)

Probability estimates.

The returned estimates for all classes are ordered by the label of classes.

For a multiclass / multinomial problem the softmax function is used to find the predicted probability of each class.

Parameters:

X : *array-like of shape (n_samples, n_features)*

Vector to be scored, where *n_samples* is the number of samples and *n_features* is the number of features.

Returns:

T : *array-like of shape (n_samples, n_classes)*

Returns the probability of the sample for each class in the model, where classes are ordered as they are in *self.classes_*.

score(X, y, sample_weight=None)

[\[source\]](#)

Return [accuracy](#) on provided data and labels.

In multi-label classification, this is the subset accuracy which is a harsh metric since you require for each sample that each label set be correctly predicted.

Parameters:

X : *array-like of shape (n_samples, n_features)*

Test samples.

y : *array-like of shape (n_samples,) or (n_samples, n_outputs)*

True labels for *X*.

sample_weight : *array-like of shape (n_samples,), default=None*

Sample weights.

Returns:

score : *float*

Mean accuracy of `self.predict(X)` w.r.t. *y*.

`set_callbacks(*callbacks)`

[\[source\]](#)

Set callbacks for the estimator.

Parameters:

****callbacks*** : *callback instances*

The callbacks to set.

Returns:

self : *estimator instance*

The estimator instance itself.

`set_fit_request(*, sample_weight: bool | None | str = '$UNCHANGED$')` → [LogisticRegression](#)

[\[source\]](#)

Configure whether metadata should be requested to be passed to the `fit` method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a [meta-estimator](#) and metadata routing is enabled with

`enable_metadata_routing=True` (see [sklearn.set_config](#)). Please check the [User Guide](#) on how the routing mechanism works.

The options for each parameter are:

- **True** : metadata is requested, and passed to **fit** if provided. The request is ignored if metadata is not provided.
- **False** : metadata is not requested and the meta-estimator will not pass it to **fit** .
- **None** : metadata is not requested, and the meta-estimator will raise an error if the user provides it.
- **str** : metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (`sklearn.utils.metadata_routing.UNCHANGED`) retains the existing request. This allows you to change the request for some parameters and not others.

 *Added in version 1.3.*

Parameters:

sample_weight : *str, True, False, or None,*
default=sklearn.utils.metadata_routing.UNCHANGED

Metadata routing for **sample_weight** parameter in **fit** .

Returns:

self : *object*

The updated object.

set_params(params)**

[\[source\]](#)

Set the parameters of this estimator.

The method works on simple estimators as well as on nested objects (such as [Pipeline](#)). The latter have parameters of the form `<component>__<parameter>` so that it's possible to update each component of a nested object.

Parameters:

****params** : *dict*

Estimator parameters.

Returns:

self : *estimator instance*

Estimator instance.

set_score_request(***, *sample_weight*: [bool](#) | [None](#) | [str](#) = '\$UNCHANGED\$') → [LogisticRegression](#) [\[source\]](#)

Configure whether metadata should be requested to be passed to the `score` method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a [meta-estimator](#) and metadata routing is enabled with

`enable_metadata_routing=True` (see [sklearn.set_config](#)). Please check the [User Guide](#) on how the routing mechanism works.

The options for each parameter are:

- `True` : metadata is requested, and passed to `score` if provided. The request is ignored if metadata is not provided.
- `False` : metadata is not requested and the meta-estimator will not pass it to `score`.
- `None` : metadata is not requested, and the meta-estimator will raise an error if the user provides it.
- `str` : metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (`sklearn.utils.metadata_routing.UNCHANGED`) retains the existing request. This allows you to change the request for some parameters and not others.

 *Added in version 1.3.*

Parameters:

sample_weight : *str, True, False, or None,*
default=sklearn.utils.metadata_routing.UNCHANGED

Metadata routing for `sample_weight` parameter in `score`.

Returns:

self : *object*

The updated object.

sparsify() [\[source\]](#)

Convert coefficient matrix to sparse format.

Converts the `coef_` member to a scipy.sparse matrix, which for L1-regularized models can be much more memory- and storage-efficient than the usual numpy.ndarray representation.

The `intercept_` member is not converted.

Warning

This method is not supported for estimators fitted with array API inputs (i.e. when `sklearn.config_context` is used with `array_api_dispatch=True`). The call may succeed but subsequent calls to `predict` and other methods involving passing arrays may raise or return unexpected results.

Returns:

`self`

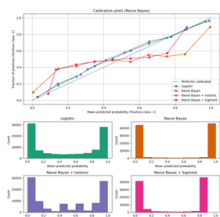
Fitted estimator.

Notes

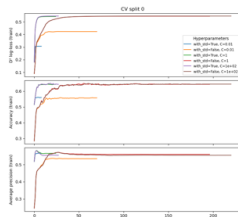
For non-sparse models, i.e. when there are not many zeros in `coef_`, this may actually *increase* memory usage, so use this method with care. A rule of thumb is that the number of zero elements, which can be computed with `(coef_ == 0).sum()`, must be more than 50% for this to provide significant benefits.

After calling this method, further fitting with the `partial_fit` method (if any) will not work until you call `densify`.

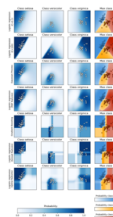
Gallery examples



Probability
Calibration curves



Analysis of the
convergence of
penalized logistic
regression models



Plot classification
probability



Column
Transformer with
Mixed Types

